
Hugo Phone

Implementation manual

and technical reference

*Recreation of the **Hugo** game (DOS, 1996) controlled by an analog telephone via DTMF tones, replicating the 1990s television experience, in a fully offline architecture based on Raspberry Pi, Asterisk and a native Rust bridge.*



Beto

Los Ángeles, Biobío, Chile

Version 1.0 ■ May 14, 2026

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Project goals	1
1.3	Design decisions	2
1.3.1	Why Raspberry Pi 4?	2
1.3.2	Why Asterisk and not something lighter?	2
1.3.3	Why Rust for the bridge?	2
1.3.4	Why DOSBox-Staging and not DOSBox-X?	2
1.4	Structure of the manual	3
1.5	Typographic conventions	3
2	System architecture	4
2.1	Overview	4
2.2	Data flow	4
2.3	Logical components	5
2.3.1	Asterisk	5
2.3.2	hugo-bridge (Rust)	5
2.3.3	DOSBox-Staging	5
2.3.4	System services	5
2.4	Network topology	5
2.5	Security decisions	6
3	Bill of materials	7
3.1	Essential components	7
3.1.1	Per-component details	7
3.2	Optional components	9
3.3	Growth plan: multi-player	9
3.4	Comparison with alternatives	9
4	Phase 1: Preparing the Raspberry Pi	10
4.1	Goal	10
4.2	Materials	10
4.3	Step 1: Flash the microSD	10
4.4	Step 2: First boot	11
4.5	Step 3: Network configuration	11
4.6	Step 4: SSH connection	11
4.7	Step 5: System update	12
4.8	Step 6: Clone the project repository	12
4.9	Step 7: Run the installer	12
4.10	Step 8: Configure secrets	13
4.11	Step 9: Verification	13

4.12	Troubleshooting	13
4.12.1	Asterisk does not start	13
4.12.2	The Pi reboots for no apparent reason or shows random lag	14
4.12.3	SSH doesn't respond	14
4.12.4	The installer fails while compiling the bridge	14
5	Phase 2: SIP test with a softphone	15
5.1	Goal	15
5.2	Step 1: Install Linphone	15
5.3	Step 2: Create a SIP account in Linphone	15
5.4	Step 3: Verify the registration from Asterisk	15
5.5	Step 4: Test call to 9000	16
5.6	Step 5: Test DTMF	16
5.7	Troubleshooting	16
5.7.1	Linphone does not register	16
5.7.2	The 9000 call drops immediately	17
5.7.3	It connects but the beep is not heard	17
5.7.4	No UserEvent appears when sending DTMF	17
5.8	Phase wrap-up	17
6	Phase 3: DOSBox-Staging and Hugo	18
6.1	Goal	18
6.2	Legal status of the game	18
6.3	Step 1: Download Hugo	18
6.4	Step 2: Verify DOSBox-Staging configuration	18
6.5	Step 3: Graphical output modes	19
6.5.1	Development mode: with a lightweight desktop	19
6.5.2	Appliance mode: direct KMSDRM	19
6.6	Step 4: Auto-start the game	19
6.6.1	Enable autologin on TTY1	19
6.6.2	Run DOSBox at login	20
6.7	Step 5: Playability test	20
6.8	Step 6: Exit cleanly	20
6.9	Troubleshooting	21
6.9.1	Hugo hangs on the intro or the cutscenes	21
6.9.2	The audio stutters	21
6.9.3	Black screen at startup	21
6.9.4	Hugo starts but doesn't respond to the keyboard	21
6.10	Phase wrap-up	21
7	Phase 4: The Rust bridge	22
7.1	Goal	22
7.2	Code structure	22
7.3	Dependencies and design philosophy	22
7.4	The AMI protocol	23
7.5	Anatomy of the <code>ami.rs</code> module	23

7.5.1	run: main loop with reconnection	23
7.5.2	connect_and_loop: connection, login and event loop	24
7.5.3	handle_event: filtering and action	24
7.5.4	read_message: wire-format parser	25
7.6	The keyboard.rs module	25
7.6.1	Creating the virtual keyboard	26
7.6.2	Injecting a press	26
7.7	The watcher.rs module	26
7.8	Practical verification	27
7.9	The complete flow with the softphone	27
7.10	Calibration	28
7.11	Enable as a service	28
7.12	Troubleshooting	28
7.12.1	Permission denied on /dev/uinput	28
7.12.2	The bridge connects but doesn't receive events	28
7.12.3	DTMFs reach the bridge but Hugo doesn't respond	28
7.12.4	Cargo can't find dependencies during compilation	29
7.13	Phase wrap-up	29
8	Phase 5: Connecting the real telephone	30
8.1	Goal	30
8.2	Background concepts	30
8.2.1	What an FXS port is	30
8.2.2	Auto-dial when going off-hook	30
8.3	Physical connections	30
8.4	Step 1: Find the HT801's IP	31
8.4.1	Method A: HT801's own IVR	31
8.4.2	Method B: router panel	31
8.5	Step 2: Access the HT801 web panel	31
8.6	Step 3: Configure the SIP account	31
8.7	Step 4: Configure Auto-Dial	32
8.8	Step 5: Verify the registration	32
8.9	Step 6: The final test	33
8.10	Troubleshooting	33
8.10.1	The phone gives no dial tone when picked up	33
8.10.2	There's dial tone but going off-hook doesn't dial 9000	33
8.10.3	It connects but the beep is not heard	33
8.10.4	DTMFs don't produce action in Hugo	34
8.10.5	The call drops on its own after a few minutes	34
9	Phase 6: Multi-player mode (future)	35
9.1	Vision	35
9.2	Hardware changes	35
9.3	Asterisk changes	35
9.3.1	sip.conf: new extensions	35

9.3.2	confbridge.conf: conference rooms	35
9.3.3	extensions.conf: queue logic	36
9.4	Turn logic	36
9.5	Game audio in the conference	36
9.5.1	Path A: ALSA loopback	36
9.5.2	Path B: PipeWire	37
9.6	Physical topology	37
9.7	Optional features	37
9.7.1	Player identification	37
9.7.2	Scoreboard / ranking	37
9.7.3	On-screen scoreboard display	37
9.7.4	Streaming to Twitch or YouTube	38
9.8	Effort estimate	38
9.9	Wrap-up	38
10	Technical reference	39
10.1	DTMF → keyboard mapping	39
10.2	Key system files	39
10.3	Useful services and commands	39
10.3.1	Managing Asterisk	39
10.3.2	Managing the bridge	40
10.3.3	Rebuilding the bridge	40
10.3.4	Inspecting the virtual keyboard	40
10.3.5	HT801 health check	40
10.4	Structure of the <code>config.toml</code> file	40
10.5	Supported key names	41
10.6	AMI protocol anatomy	41
10.6.1	Initial banner	41
10.6.2	Login	41
10.6.3	Successful login response	41
10.6.4	UserEvent produced by the dialplan	42
10.7	Glossary	42
10.8	External references	42
10.9	Credits	43

Chapter 1

Introduction

1.1 Motivation

In the early 1990s, several television channels aired children’s programs where viewers controlled an on-screen character *live* through the keypad of their telephone. The most famous was **Hugo**, a small Norwegian troll created by the Danish company ITE Media, who had to rescue his family (Hugolina and their children) from the evil witch Scylla. The broadcast reached more than forty countries, and in Chile it aired on several free-to-air channels.

The mechanism was simple in appearance and sophisticated in infrastructure: the show’s production company set up a paid phone line, children would call in, wait their turn while listening to music and the game’s audio, and once their turn came they would control Hugo for about a minute by pressing keypad digits:

- **2** to jump or go up
- **4** to move left
- **6** to move right
- **8** to crouch or go down
- **5** for special actions

Each press generated a *DTMF* (*Dual-Tone Multi-Frequency*) tone that the studio’s switchboard decoded, translated into a game command, and sent to the PC running the interactive version of Hugo. The latency —comparable to that of a football match streamed today— was part of the charm and the challenge.

Thirty years later, every component of that system is available as free software or low-cost hardware, which makes it possible to reconstruct the full experience at home, offline, and at a fraction of the original cost. This manual documents step by step how to do it.

1.2 Project goals

1. Run **Hugo** (DOS version, 1996) on a Raspberry Pi 4 (2 GB) with HDMI output to a television.
2. Connect a physical **analog DTMF telephone** to the system through an FXS adapter (Grandstream HT801).
3. Implement a **local telephone exchange** on the same Pi using Asterisk, without requiring Internet access or external telephony services.

4. Develop a **bridge** in Rust that translates the DTMF tones captured by Asterisk into keystrokes injected into the DOSBox-Staging emulator.
5. Document the system in enough detail that it can be replicated, maintained, and extended to a *multi-player* TV-style scenario.

1.3 Design decisions

1.3.1 Why Raspberry Pi 4?

The Pi 4 is powerful enough to run Asterisk, DOSBox-Staging and the bridge at the same time, while remaining a compact, silent, low-power (5–7 W) *appliance* with direct HDMI output. The 2 GB model is the cheapest option available and still leaves RAM headroom after all services are loaded.

1.3.2 Why Asterisk and not something lighter?

Asterisk is the most widely used and best documented PBX implementation in the free software ecosystem. It ships with native DTMF decoding (RFC 2833 and *inband*), an extensible *dialplan*, the *Manager Interface* (AMI) for integration with external software, and *ConfBridge* support for *multiparty* conferences. Alternatives like *baresip* or *kamailio* are lighter, but they don't include the high-level DTMF decoding or the *dialplan* needed here.

1.3.3 Why Rust for the bridge?

The *bridge* is the component that receives the DTMF events from Asterisk and translates them into virtual keystrokes through the Linux kernel's `uinput` subsystem. The desirable properties are:

- **Minimal and predictable latency:** the user perceives any delay between pressing the phone keypad and seeing the character move.
- **Low RAM consumption:** the 2 GB Pi has little headroom once DOSBox is loaded.
- **Stability:** the service must run indefinitely without restarts.
- **No runtime dependencies:** ideally a single binary, with no virtual environments or interpreters.

Rust meets all four properties effortlessly. The final binary weighs a few megabytes and consumes around 3–5 MB of RAM at runtime.

1.3.4 Why DOSBox-Staging and not DOSBox-X?

DOSBox-Staging is a modern *fork* with performance improvements on ARM platforms, active maintenance, and compatibility with most DOS games. For Hugo (1996, a simple engine based on VGA and Sound Blaster) any of the *forks* would work, but *Staging* is currently the best-maintained option for Raspberry Pi.

1.4 Structure of the manual

The document is organized in three blocks:

Conceptual block (chapters 1–3): introduction, overall system architecture, and detailed list of materials (*Bill of Materials*).

Implementation block (chapters 4–8): six sequential phases, each with a clear goal, verifiable steps and *troubleshooting*. Each phase builds on the previous one.

Reference block (chapters 9–10): forward-looking view of multi-player mode and complete technical reference (AMI protocols, DTMF mapping, useful commands).

Note

The phases are designed for incremental validation. If a phase fails, you should not move on to the next one: the system only becomes harder to debug as components stack up on a base that doesn't yet work.

1.5 Typographic conventions

- `Shell commands` and file names are shown in monospaced font.
- *Technical terms* introduced for the first time appear in italics.
- Callout boxes follow this scheme:

Note

Note boxes provide useful but non-critical additional context.

Warning

Warning boxes flag situations where a mistake can break the system or require a reinstall.

Tip

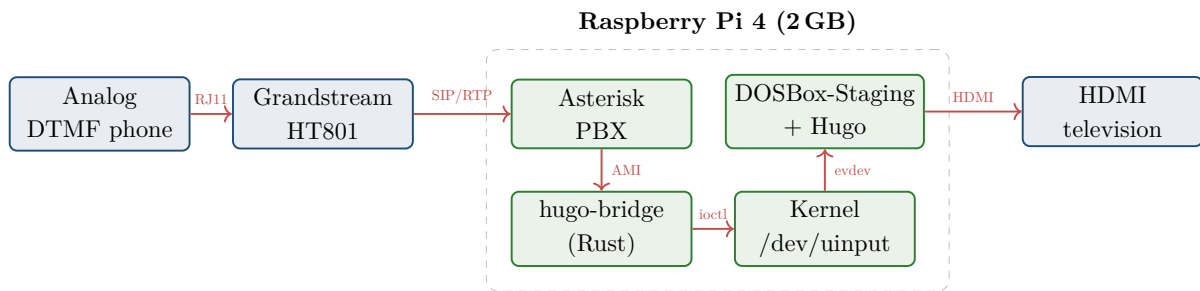
Tip boxes contain shortcuts, tricks, or lessons learned from common mistakes.

Chapter 2

System architecture

2.1 Overview

The complete system is made up of three physical domains and five logical domains coexisting inside a single Raspberry Pi:



2.2 Data flow

Pressing the digit **2** on the phone travels along the following path:

1. The phone generates the DTMF tone matching **2** (697 Hz + 1336 Hz combined) and sends it as analog audio to the HT801's FXS port.
2. The HT801 detects the tone and, depending on its configuration (*DTMF method: RFC 2833*), encapsulates it as an *out-of-band* RTP event and forwards it over SIP to Asterisk.
3. Asterisk receives the RFC 2833 event and exposes it to the running *dialplan* as a digit read by the `Read()` function.
4. The *dialplan* fires a `UserEvent` named `HugoKey` with the `Key=2` field, emitted by the *Asterisk Manager Interface* (AMI) over TCP on `127.0.0.1:5038`.
5. The `hugo-bridge` process, subscribed to AMI, parses the event, looks up the configured *keymap* and obtains the target key (`KEY_UP`).
6. Through `ioctl(2)` on `/dev/uinput`, the bridge simulates a physical press (press + sync + sleep + release + sync) on a previously created virtual keyboard.
7. The kernel propagates the event through `evdev` to the input subsystem, where it is read by the SDL2 library that DOSBox-Staging uses for input.
8. DOSBox-Staging translates the key code into an IBM PC keyboard interrupt 16h, which Hugo reads inside its *event loop*.

9. Hugo updates the character's position and renders the next frame through emulated VGA, which DOSBox dumps to the HDMI screen.

The total end-to-end latency is dominated by the local SIP network and the game's render loop. In internal tests it ranges between 5 and 30 ms, perfectly playable.

2.3 Logical components

2.3.1 Asterisk

Open-source PBX. Here it plays four roles:

- **SIP registrar:** accepts the HT801 registering as extension 101, and optionally softphones (Linphone) as test extensions.
- **DTMF decoding:** extracts digits from RFC 2833 or *inband* and delivers them to the dialplan.
- **Dialplan:** program that defines what happens with each call. In Hugo Phone it is an infinite loop that reads a digit and emits a `UserEvent`.
- **Manager Interface (AMI):** TCP socket at 127.0.0.1:5038 where external clients authenticate and receive events in real time.

2.3.2 hugo-bridge (Rust)

Native binary written in Rust, running as a `systemd` service. Internal components:

- **In-house AMI client:** TCP/text implementation of the Asterisk Manager protocol, with no third-party *crate* dependencies. About 150 lines of code in `src/ami.rs`.
- **Virtual keyboard:** *wrapper* around the `uinput` crate, creates an input device that the kernel exposes with the name `hugo-phone-virtual-keyboard`.
- **Configuration hot-reload:** *watcher* based on `notify` that detects changes to the `config.toml` file and updates the *keymap* and other parameters without restarting the service.
- **Automatic reconnection:** if the connection to Asterisk drops, the bridge retries with exponential *backoff* up to 30 seconds.

2.3.3 DOSBox-Staging

DOS emulator optimized for modern hardware. Configured in *fullscreen* mode over KMSDRM (no X server) to minimize resource usage. It launches Hugo automatically from the `[autoexec]` section of its configuration file.

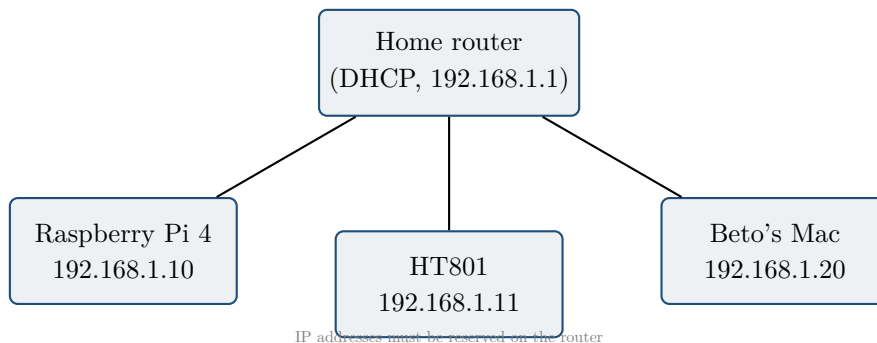
2.3.4 System services

2.4 Network topology

The complete system lives on a home LAN with three devices that communicate:

Service	Start mechanism	Function
asterisk.service	systemd	SIP PBX
hugo-bridge.service	systemd (after asterisk)	DTMF → keyboard
DOSBox-Staging	.bashrc on TTY1	Runs Hugo
Autologin tty1	raspi-config	Boot without keyboard

Table 2.1: Persistent services on the Raspberry Pi.

**Tip**

Reserving static IPs on the router (tied to each device’s MAC address) is essential. If the HT801 changes IP between reboots, Asterisk won’t be able to send it packets and the line will go mute.

2.5 Security decisions

The system is designed for a trusted home network:

- Asterisk does *not* expose its 5060 port to the outside. The router must **not** port-forward it.
- AMI only listens on 127.0.0.1, accessible only to local processes.
- SIP and AMI passwords are random and changed at install time (the placeholder values “cambiame-*” must be replaced).
- The bridge user runs as **root** because it needs access to `/dev/uinput`. Alternatively, the `input` group with a `udev` rule can be used.

Warning

If for some reason you decide to expose the system to the Internet (for example, to let friends connect remotely), you must use SIP over TLS, strong passwords and, preferably, a VPN. Plain-text SIP on the open Internet is one of the most common attack vectors for telephone fraud.

Chapter 3

Bill of materials

3.1 Essential components

The following components are indispensable for the system in its minimum configuration (a single phone). Prices are indicative, in Chilean pesos as of mid-2026.

Component	Suggested model	Price (CLP)
Single Board Computer	Raspberry Pi 4 Model B (2 GB)	\$85,000
Power supply	Official Raspberry Pi USB-C 15 W (5 V/3 A)	\$15,000
Storage	MicroSD SanDisk 32 GB Class 10 / A1	\$10,000
Thermal dissipation	Case with fan for Pi 4	\$12,000
Video cable	micro-HDMI to HDMI cable, 1.5 m	\$5,000
Telephone adapter	Grandstream HT801 (1 FXS port)	\$40,000
Telephone cable	RJ11 cable, 2 m	\$2,000
Network cable	Ethernet Cat5e patch cord, 1 m	\$3,000
Analog telephone	Any landline phone with DTMF (1990s+)	—
Television/monitor	Any device with HDMI input	—
Essentials total		~\$172,000

Table 3.1: Essential bill of materials.

3.1.1 Per-component details

Raspberry Pi 4 (2 GB)

Available in Chile through official distributors such as `raspberrypi.cl` and `mcielectronics.cl`. The Pi 4B model with at least 2 GB of RAM is required. Older models (3B+ or below) don't have enough emulation performance, and models with less RAM leave the system with no headroom.

Note

If the Pi 4 (2 GB) is out of stock, the Pi 5 (2 GB) works as well and performs better. The price difference is usually around 10%. Either one meets the requirements.

Official USB-C power supply

Critical. The Pi 4 demands a steady 5 V at 3 A under load. Mobile phone chargers, even branded ones, often deliver unstable voltage, which causes *undervoltage warnings*, spontaneous reboots and microSD corruption. The official supply is the only reliable way to avoid it.

MicroSD

32 GB is more than enough. The important specifications are:

- **Class 10:** guarantees 10 MB/s sustained write.
- **A1 or A2 endurance:** optimized for applications (not just large files such as video).

Recommended brands: SanDisk Endurance, Samsung Evo Plus.

Tip

Cheap unbranded microSDs fail within months when an operating system writes constantly. For an always-on *appliance*, it's worth spending a little more.

Grandstream HT801

An *Analog Telephone Adapter* (ATA) with one FXS (*Foreign eXchange Subscriber*) port, that is, a port that supplies line voltage (~ 48 V DC) and ringing voltage (~ 90 V AC) to an analog telephone, and on the other side connects to the local Ethernet network and registers as a SIP client.

Relevant features:

- G.711 (PCMU/PCMA), G.722, G.729, iLBC, OPUS codecs
- In-band DTMF, RFC 2833 and SIP INFO
- Web-configurable
- *Off-hook Auto-Dial* feature: automatically dials a number when the handset is lifted

In Chile it's sold via MercadoLibre, VoIP distributors such as Inttelec or Centralinet, or by direct import from Aliexpress (with warranty risk).

Analog telephone

Any landline phone with a DTMF keypad from the last forty years works. **Rotary-dial phones don't work** because they generate pulses, not tones. Classic models that fit the role and add 1990s aesthetic:

- Panasonic KX-T series

- General Electric with curly cord
- Early cordless VTech (with a large base)
- Any “hotel” phone Cetis or similar

3.2 Optional components

Component	Reason	Price (CLP)
Small used CRT TV	Authentic 1990s aesthetic	\$20,000–40,000
5-port Ethernet switch	If the router is far from the setup	\$10,000
USB HDMI capture card	To record/stream gameplay	\$25,000
External 3.5 mm speakers	If the TV has weak audio	\$15,000
Decorative labels	“Player 1” style design on the phone	\$3,000

Table 3.2: Optional components.

3.3 Growth plan: multi-player

The base system supports a single phone. To scale up to the TV mode with multiple simultaneous players:

- Replace the HT801 with an **HT802** (2 FXS ports, ~\$55,000) or **HT814** (4 FXS ports, ~\$110,000).
- Acquire additional phones.
- Reconfigure Asterisk with ConfBridge and turn-queue logic.

This growth is documented in detail in chapter 9.

3.4 Comparison with alternatives

	Pi 4 (2 GB)	Pi 5 (2 GB)	Used x86 mini PC
Approx. price (CLP)	\$85,000	\$95,000	\$100,000
Hugo emulation	Good	Very good	Excellent (native)
Power draw	5–7 W	8–12 W	15–25 W
Asterisk	Perfect	Perfect	Perfect
Physical size	Credit card	Credit card	Small
HDMI to TV	Yes (micro)	Yes (micro)	Yes (standard)
Appliance look	Excellent	Excellent	Good

Table 3.3: Platform comparison for Hugo Phone.

For this project the **Pi 4 (2 GB)** is selected for its combination of price, size and power efficiency.

Chapter 4

Phase 1: Preparing the Raspberry Pi

4.1 Goal

By the end of this phase, the Raspberry Pi must be:

- Booting Raspberry Pi OS Lite (64-bit).
- Reachable over SSH from the working Mac.
- Assigned a static IP reserved on the router.
- Running Asterisk 20 and responding on its CLI.
- With the project's configuration files placed in `/etc/asterisk/`.

4.2 Materials

- Raspberry Pi 4 (2 GB)
- 32 GB microSD
- Official USB-C power supply
- micro-HDMI cable
- USB keyboard (only for the first boot)
- Television or HDMI monitor
- Mac (or other computer) with microSD reader

4.3 Step 1: Flash the microSD

From the Mac, download and install the *Raspberry Pi Imager* from the official site [raspberrypi.com/software](https://www.raspberrypi.com/software). Connect the microSD to the Mac (with a USB adapter if necessary).

When the Imager opens, configure:

- **Device:** Raspberry Pi 4
- **Operating System:** Raspberry Pi OS Lite (64-bit)
- **Storage:** the inserted microSD

Before clicking *Write*, click the gear icon (**Edit Settings**) to configure:

- **Hostname:** hugo-pbx

- **User:** beto (or whatever name you prefer), with a strong password.
- **SSH:** enabled, password or public-key authentication.
- **WiFi:** SSID and password (only if you won't use Ethernet).
- **Locale:** America/Santiago, latam keyboard.

Confirm and let the Imager write (between 5 and 15 minutes depending on the reader's speed).

4.4 Step 2: First boot

Insert the microSD into the Pi, plug in HDMI, the USB keyboard and, finally, the power supply. The first boot takes between 1 and 2 minutes because it automatically resizes the filesystem. When it finishes, a *login prompt* will be shown.

Log in with the username and password configured in the previous step.

4.5 Step 3: Network configuration

The desirable setup is a static IP reserved on the router for the Pi's MAC address. To obtain the MAC of the Ethernet port:

```
1 ip link show eth0 | grep ether
```

The output will look like `link/ether dc:a6:32:xx:xx:xx`. Note that MAC, enter the router's admin panel (usually `http://192.168.1.1`) and reserve the IP `192.168.1.10` for that MAC.

Reboot the Pi and verify the IP:

```
1 ip addr show eth0
```

Tip

If the router hands out IPs from another range (e.g. `192.168.0.x` or `10.0.0.x`), adapt every address in the manual accordingly.

4.6 Step 4: SSH connection

From the Mac:

```
1 ssh beto@192.168.1.10
```

If the router resolves mDNS, this also works:

```
1 ssh beto@hugo-pbx.local
```

From this point on the USB keyboard and HDMI cable are no longer needed.

4.7 Step 5: System update

```
1 sudo apt update
2 sudo apt upgrade -y
3 sudo reboot
```

Reconnect over SSH after the reboot.

4.8 Step 6: Clone the project repository

Copy the `hugo-phone/` directory delivered in the project repository onto the Pi. If it's in a Git service:

```
1 cd ~
2 git clone <repository-URL> hugo-phone
3 cd hugo-phone
```

If the files are only on the Mac:

```
1 # From the Mac
2 scp -r hugo-phone/ beto@hugo-pbx.local:~/
```

4.9 Step 7: Run the installer

```
1 cd ~/hugo-phone
2 sudo bash scripts/install-all.sh
```

The script performs, in order:

1. Full `apt update` && `apt upgrade`.
2. Installation of Asterisk 20, DOSBox-Staging, `build-essential`, `libudev-dev` and other dependencies.
3. Copies the `sip.conf`, `extensions.conf` and `manager.conf` files to `/etc/asterisk/` with the correct permissions.
4. Installation of *rustup* for the `beto` user (downloads and installs the stable toolchain).
5. Compilation of the bridge in `release` mode, with `LTO` and `strip`.
6. Creation of the `udev` rule for `/dev/uinput` and loading of the kernel module.
7. Installation of the `systemd` service `hugo-bridge.service`.
8. Deployment of the DOSBox-Staging configuration to `~/.config/dosbox/`.

The slowest phase is compiling the bridge, which on a Pi 4 takes between 5 and 8 minutes. Estimated total time: 20–30 minutes depending on network speed.

4.10 Step 8: Configure secrets

The initial passwords are placeholders and must be replaced before any real use:

```
1 sudo nano /etc/asterisk/sip.conf
2 sudo nano /etc/asterisk/manager.conf
3 sudo nano /opt/hugo-phone/bridge/config.toml
```

Replace every occurrence of `cambiame-101`, `cambiame-102` and `cambiame-ami`.

Warning

The password in `manager.conf` (section `[hugo-bridge]`) and the one in `config.toml` (section `[ami]`) must be **identical**. Otherwise the bridge won't be able to authenticate against Asterisk.

Apply the changes by restarting Asterisk:

```
1 sudo systemctl restart asterisk
```

4.11 Step 9: Verification

Enter the interactive Asterisk console:

```
1 sudo asterisk -rvvv
```

Inside the CLI, run:

```
hugo-pbx*CLI> sip show peers
```

It must list extensions 101 and 102 (the latter is used in the next phase) with status `Unmonitored` (no clients registered yet).

```
hugo-pbx*CLI> manager show settings
```

It must show `Enabled: Yes` and `Port: 5038`.

```
hugo-pbx*CLI> dialplan show hugo-game
```

It must display extension 9000 with its `Answer`, `Wait`, `Playback(beep)`, `Read` and `UserEvent` steps.

If all three commands respond without errors, **Phase 1 is complete**.

4.12 Troubleshooting

4.12.1 Asterisk does not start

```
1 sudo journalctl -u asterisk -n 80
```

It is almost always a syntax error in one of the `.conf` files, which the log points out with an exact line number.

4.12.2 The Pi reboots for no apparent reason or shows random lag

The power supply is probably insufficient. The Pi shows a yellow lightning bolt icon on screen when it detects *undervoltage*. Switch to the official 5 V/3 A power supply.

4.12.3 SSH doesn't respond

- Check that the Ethernet cable is plugged in or that WiFi has properly joined the network. Plug in keyboard + HDMI and run `ip addr`.
- Check that the SSH service is running: `sudo systemctl status ssh`.
- Confirm that the router's firewall isn't blocking LAN traffic.

4.12.4 The installer fails while compiling the bridge

- Retry manually as the `beto` user:

```
1 cd /opt/hugo-phone/bridge
2 ~/.cargo/bin/cargo build --release
```

- If it fails due to lack of memory (unlikely on a Pi 4 2 GB, but possible), add *swap*:

```
1 sudo dphys-swapfile swapoff
2 sudo sed -i 's/CONF_SWAPSIZE=.* /CONF_SWAPSIZE=2048/' \
3   /etc/dphys-swapfile
4 sudo dphys-swapfile setup
5 sudo dphys-swapfile swapon
```

Chapter 5

Phase 2: SIP test with a softphone

5.1 Goal

Validate the Asterisk PBX before touching the HT801. To do so, a *softphone* (Linphone) is registered as extension 102, the number 9000 is dialed, and we confirm that the DTMF tones sent from the app reach Asterisk as AMI events.

This step is important because it lets us verify the software side of the system without yet depending on the analog hardware, simplifying troubleshooting.

5.2 Step 1: Install Linphone

Linphone is a free, cross-platform SIP client:

- **iOS:** App Store, search for Linphone.
- **Android:** Google Play, search for Linphone.
- **macOS:** linphone.org/technical-corner/linphone.

Install the app on any of the available devices that share the same local network as the Pi.

5.3 Step 2: Create a SIP account in Linphone

Open Linphone and enter the wizard: Assistant → Use a SIP account (or Add Account, depending on the version).

Fill in:

Field	Value
Username	102
Password	the secret defined for extension 102 in <code>sip.conf</code>
Domain	192.168.1.10 (the Pi's IP)
Transport	UDP
Display Name	Test Hugo

Confirm and verify that the status indicator turns green with the legend **Registered** or **Connected**.

5.4 Step 3: Verify the registration from Asterisk

On the Pi:

```
1 sudo asterisk -rvvv
```

Inside the CLI:

```
hugo-pbx*CLI> sip show peers
Name/username      Host                Status
101/101            (Unspecified)      UNKNOWN
102/102            192.168.1.20:xxxx  OK (3 ms)
```

Extension 102 must appear with the IP of the device running Linphone and status OK.

5.5 Step 4: Test call to 9000

From Linphone, dial 9000 and press the call button.

Two things should be observed simultaneously:

1. In the device's earpiece: a short *beep*, produced by the `Playback(beep)` step in the dialplan.
2. In the Asterisk console: the dialplan execution trace.

```
-- Executing [9000@hugo-game:1] NoOp("PJSIP/102-...", "Hugo Phone - ...")
-- Executing [9000@hugo-game:2] Answer("PJSIP/102-...", "")
-- Executing [9000@hugo-game:3] Wait("PJSIP/102-...", "1")
-- Executing [9000@hugo-game:4] Playback("PJSIP/102-...", "beep")
-- <PJSIP/102-...> Playing 'beep.gsm' (language 'en')
-- Executing [9000@hugo-game:5] Read("PJSIP/102-...", "TECLA,,1,,3600")
```

5.6 Step 5: Test DTMF

While the call is still active, in Linphone open the tone keypad and press the digit **2**.

In the Asterisk console you should see:

```
== Manager 'hugo-bridge' logged on from 127.0.0.1
== UserEvent: HugoKey
Channel: PJSIP/102-00000001
Caller: 102
Key: 2
-- Executing [9000@hugo-game:6] GotoIf(...)
-- Executing [9000@hugo-game:5] Read("PJSIP/102-...", "TECLA,,1,,3600")
```

Repeat with **4**, **6**, **8** and **5**. Each must trigger its own `UserEvent`.

If the events appear, **Phase 2 is complete**: the PBX captures DTMF correctly and the bridge will be able to read them.

5.7 Troubleshooting

5.7.1 Linphone does not register

- Check that the Pi is reachable from the device: `ping 192.168.1.10`.

- Confirm that UDP port 5060 isn't blocked by a firewall on the Pi (Raspberry Pi OS Lite ships with no firewall enabled by default).
- Look at Asterisk logs for failed attempts:

```
1 sudo asterisk -rvvv
2 hugo-pbx*CLI> sip set debug on
```

5.7.2 The 9000 call drops immediately

- Check that extension 102 is assigned to context `hugo-game` in `sip.conf`.
- Run `dialplan show hugo-game` in the CLI and confirm that extension 9000 is present.

5.7.3 It connects but the beep is not heard

- Typical NAT or incompatible-codec problem. Verify that `allow=ulaw` and `allow=alaw` are in `sip.conf` and that Linphone also supports these codecs in its advanced configuration.
- Check the device's volume.

5.7.4 No UserEvent appears when sending DTMF

- Linphone may send DTMF as *inband* tones instead of RFC 2833. In Settings → Audio → DTMF mode, force the RFC 2833 or SIP INFO method.
- Confirm that the `sip.conf` file has the line `dtmfmode=rfc2833` for extension 102.

5.8 Phase wrap-up

At this point the following is validated:

- Asterisk accepts SIP clients and registers them correctly.
- The dialplan answers calls and plays audio.
- The `Read()` function captures DTMF into a variable.
- The `UserEvent` is emitted and visible on AMI.

That is everything the bridge needs from the Asterisk side. The next phase prepares DOSBox-Staging and Hugo so that the bridge has somewhere to inject the keys.

Chapter 6

Phase 3: DOSBox-Staging and Hugo

6.1 Goal

Have Hugo running on the Pi, displayed on the HDMI screen and playable with a USB keyboard. This is the last individual piece before tying everything together in Phase 4.

6.2 Legal status of the game

Hugo is *abandonware*: commercial software whose support by the original publisher ceased decades ago, and whose passive distribution is broadly tolerated though not formally authorized. Sites like myabandonware.com and abandonwaregames.net host downloadable copies. The legal situation varies by jurisdiction, but home and educational use has not been the subject of litigation.

6.3 Step 1: Download Hugo

From the Mac, open in a browser:

```
https://www.myabandonware.com/game/hugo-eya
```

Download the ZIP file the site provides. Once in ~/Downloads/, transfer it to the Pi:

```
1 scp ~/Downloads/Hugo*.zip beto@hugo-pbx.local:/tmp/
```

On the Pi, unzip into the game's directory:

```
1 sudo mkdir -p /opt/hugo-phone/games/hugo
2 sudo unzip /tmp/Hugo*.zip -d /opt/hugo-phone/games/hugo
3 sudo chown -R beto:beto /opt/hugo-phone/games/hugo
4 ls /opt/hugo-phone/games/hugo/
```

You should see `hugo.exe` (or a similar file). Some distributions ship an `install.exe` installer that must be run inside DOSBox the first time.

6.4 Step 2: Verify DOSBox-Staging configuration

The install script copied a `dosbox-staging.conf` optimized for Hugo onto the Pi. Verify:

```
1 cat ~/.config/dosbox/dosbox-staging.conf | head -30
```

If the game's executable is not exactly `hugo.exe`, edit the `[autoexec]` section:

```
1 nano ~/.config/dosbox/dosbox-staging.conf
```

The section to tweak is:

```
[autoexec]
mount c /opt/hugo-phone/games
c:
cd hugo
hugo.exe
exit
```

6.5 Step 3: Graphical output modes

DOSBox-Staging can draw in several ways. In order of increasing simplicity:

6.5.1 Development mode: with a lightweight desktop

For tests and diagnostics, install Openbox (a minimalist window manager):

```
1 sudo apt install -y xinit openbox
2 echo "exec openbox-session" > ~/.xinitrc
3 startx /usr/games/dosbox-staging
```

This starts an X11 server with Openbox and launches DOSBox-Staging inside. Useful for experimentation.

6.5.2 Appliance mode: direct KMSDRM

For the final operation, DOSBox-Staging can draw directly to the kernel framebuffer with no X server, which reduces memory usage and removes compositing latency:

```
1 SDL_VIDEODRIVER=kmsdrm dosbox-staging
```

Note

KMSDRM only works from a local TTY (not over SSH X-forwarding). It is the recommended mode for normal appliance operation.

6.6 Step 4: Auto-start the game

Configure the Pi so that powering on takes you directly into Hugo, with no user intervention.

6.6.1 Enable autologin on TTY1

```
1 sudo raspi-config
```

Navigate to: System Options → Boot / Auto Login → Console Autologin. Save and exit.

6.6.2 Run DOSBox at login

Edit the `.bashrc` of the `beto` user:

```
1 nano ~/.bashrc
```

Append at the end:

```
if [ -z "$DISPLAY" ] && [ "$(tty)" = "/dev/tty1" ]; then
    SDL_VIDEODRIVER=kmsdrm dosbox-staging
fi
```

The condition prevents DOSBox from running if you log in over SSH or from a TTY other than 1.

After reboot, the sequence should be:

1. Kernel boot
2. Autologin as `beto` on `tty1`
3. DOSBox-Staging running in KMSDRM mode
4. Hugo auto-launched per `[autoexec]`

6.7 Step 5: Playability test

With a USB keyboard plugged into the Pi and Hugo loaded, test:

- Arrow keys → character movement
- Space bar → action
- Esc → menu/exit

If Hugo feels excessively fast or slow, adjust the `cycles` line in `dosbox-staging.conf`:

```
1 [cpu]
2 cycles = fixed 8000    # Increase if too slow, decrease if too fast
```

The original Hugo required roughly a 386SX at 25 MHz. Start with `cycles = fixed 8000` and calibrate.

6.8 Step 6: Exit cleanly

Two Esc presses bring up Hugo's menu. The *Quit* option returns to the DOSBox prompt; the `exit` command in the `[autoexec]` block closes DOSBox automatically.

For an emergency exit (for example if the game hangs): `Ctrl+F9` is the *kill switch* for DOSBox-Staging.

6.9 Troubleshooting

6.9.1 Hugo hangs on the intro or the cutscenes

DOSBox-Staging on ARM occasionally has problems decoding VGA animations. Pressing any key to skip the intro usually solves it. If it persists, lower the output resolution:

```
1 [sdl]
2 windowresolution = 640x480
```

6.9.2 The audio stutters

Increase the mixer buffer:

```
1 [mixer]
2 prebuffer = 50
3 blocksize = 2048
```

If stuttering remains, lower the quality: `rate = 22050`.

6.9.3 Black screen at startup

The KMSDRM driver is probably not available in the installed SDL2 version. Alternatives:

```
1 SDL_VIDEODRIVER=fbcon dosbox-staging
```

Or install the minimal desktop:

```
1 sudo apt install -y xinit openbox
```

And launch DOSBox inside `startx`.

6.9.4 Hugo starts but doesn't respond to the keyboard

- Verify that `core = dynamic` is present in the `[cpu]` section.
- Some titles require `cputype = 386` instead of `486_slow`.

6.10 Phase wrap-up

At this point the game runs with a USB keyboard. The next phase replaces that USB keyboard with a virtual one controlled from the phone.

Phase 4: The Rust bridge

7.1 Goal

Bring up the `hugo-bridge` bridge, which wires Asterisk's AMI events into the kernel's uinput subsystem, translating DTMF into virtual keystrokes.

This is the most software-heavy phase of the project. It is also used to document the bridge's internal architecture in detail.

7.2 Code structure

```
bridge/
|-- Cargo.toml           # Crate manifest
|-- config.toml          # Runtime configuration
|-- hugo-bridge.service  # systemd unit
`-- src/
    |-- main.rs          # Entry point, tokio runtime
    |-- config.rs        # TOML parsing with serde
    |-- ami.rs           # Minimal AMI client
    |-- keyboard.rs      # uinput virtual keyboard
    `-- watcher.rs       # File watcher (hot-reload)
```

7.3 Dependencies and design philosophy

The bridge was designed to minimize external dependencies. The only *crates* in `Cargo.toml` are:

- `tokio`: async runtime (with specific *features*, not `full`).
- `serde` + `toml`: configuration-file parsing.
- `tracing` + `tracing-subscriber`: structured logging.
- `clap`: command-line argument parsing.
- `uinput`: *wrapper* over the `ioctl` API of `/dev/uinput`.
- `notify`: file watcher for hot-reload.

Conspicuously absent: any existing AMI client. The protocol is simple enough to implement by hand in about 150 lines, avoiding extra dependencies and keeping control over every byte that crosses the socket.

7.4 The AMI protocol

Asterisk Manager Interface is a plain-text protocol over TCP. Its informal specification is:

- TCP connection to 127.0.0.1:5038.
- The server sends a banner: `Asterisk Call Manager/X.Y.Z\r\n`.
- Messages (in both directions) consist of `Key: Value` pairs separated by `\r\n`, terminated by a blank line (`\r\n\r\n`).
- The client must authenticate with an `Action: Login` message including `Username` and `Secret` fields.
- After that, the server sends asynchronous events whenever they occur.

A real-world message example:

```
Event: UserEvent
Privilege: user,all
UserEvent: HugoKey
Channel: PJSIP/102-00000001
Caller: 102
Key: 2
Uniqueid: 1715641234.1
(blank line)
```

7.5 Anatomy of the `ami.rs` module

The AMI client is implemented in four functions:

7.5.1 `run`: main loop with reconnection

```
1 pub async fn run(mut cfg_rx: watch::Receiver<Arc<Config>>) -> Result<()> {
2     let initial_cfg = cfg_rx.borrow().clone();
3     let mut keyboard = VirtualKeyboard::new(
4         &initial_cfg.keymap,
5         initial_cfg.bridge.press_duration_ms,
6     )?;
7
8     let mut backoff_secs = 1u64;
9     loop {
10        let cfg = cfg_rx.borrow().clone();
11        match connect_and_loop(&cfg, &mut keyboard, &mut cfg_rx).await {
12            Ok(()) => {
13                backoff_secs = 1;
14            }
15            Err(e) => {
16                error!("AMI error: {}. Retrying in {}s...", e, backoff_secs);
17                sleep(Duration::from_secs(backoff_secs)).await;
18                backoff_secs = (backoff_secs * 2).min(30);
19            }
20        }
21    }
22 }
```

```

20     }
21 }
22 }

```

Three key decisions:

- **The keyboard is created only once**, outside the connection loop. Recreating it between reconnections causes a brief window where the `/dev/input/eventXX` device disappears, which SDL2 handles poorly.
- **Exponential backoff** between retries (1, 2, 4, 8, 16, 30, 30, ...), capped at 30 seconds so we don't wait minutes when Asterisk restarts.
- **The config *receiver* is passed by mutable reference** so the inner function can detect configuration changes while connected.

7.5.2 connect_and_loop: connection, login and event loop

After connecting the TCP socket, reading the banner and sending Action: Login, the main loop uses `tokio::select!` to multiplex two event sources:

```

1  loop {
2      tokio::select! {
3          msg = read_message(&mut reader) => {
4              let msg = msg?;
5              handle_event(&msg, &event_name, &key_field, keyboard).await;
6          }
7          changed = cfg_rx.changed() => {
8              if changed.is_err() { return Ok(()); }
9              let new_cfg = cfg_rx.borrow().clone();
10             keyboard.update_keymap(&new_cfg.keymap);
11             keyboard.set_press_duration(new_cfg.bridge.press_duration_ms);
12             if new_cfg.ami != cfg.ami {
13                 return Ok(()); // forces reconnection
14             }
15         }
16     }
17 }

```

The `select!` blocks until one of two things happens. If a message arrives, it is processed. If the configuration changed (hot-reloaded by the watcher), it is applied to the keyboard, and if the AMI parameters changed, a reconnection is forced by returning `Ok(())` to the outer loop.

7.5.3 handle_event: filtering and action

```

1  async fn handle_event(
2      msg: &HashMap<String, String>,
3      event_name: &str,
4      key_field: &str,
5      keyboard: &mut VirtualKeyboard,
6  ) {

```

```

7     if msg.get("Event").map(String::as_str) != Some("UserEvent") { return; }
8     if msg.get("UserEvent").map(String::as_str) != Some(event_name) { return;
9         }
10
11     let Some(dtmf) = msg.get(key_field) else { return };
12     let caller = msg.get("Caller").map(String::as_str).unwrap_or("?");
13
14     info!("DTMF {} from {}", dtmf, caller);
15     if let Err(e) = keyboard.press_dtmf(dtmf).await {
16         error!("Failed to press key: {}", e);
17     }
18 }

```

Three successive filters discard AMI events the bridge doesn't care about. Only `UserEvents` of type `HugoKey` (configurable) reach the key-injection path.

7.5.4 read_message: wire-format parser

```

1  async fn read_message<R: tokio::io::AsyncBufRead + Unpin>(
2      reader: &mut R,
3  ) -> std::result::Result<HashMap<String, String>, BoxError> {
4      let mut map = HashMap::new();
5      let mut line = String::new();
6      loop {
7          line.clear();
8          let n = reader.read_line(&mut line).await?;
9          if n == 0 {
10             return Err("AMI connection closed by server".into());
11          }
12          let trimmed = line.trim_end_matches(['\r', '\n']);
13          if trimmed.is_empty() {
14              if map.is_empty() { continue; }
15              return Ok(map);
16          }
17          if let Some((k, v)) = trimmed.split_once(':') {
18              map.insert(k.trim().to_string(), v.trim().to_string());
19          }
20      }
21  }

```

Reads line by line until it finds a blank line, at which point it considers the message complete. Malformed lines (without a :) are silently discarded, typical behavior of tolerant parsers in legacy protocols.

7.6 The keyboard.rs module

The Linux kernel's `uinput` subsystem lets a *user space* process create virtual input devices. The kernel exposes them just like any physical keyboard under `/dev/input/eventXX` and applications like `DOSBox-Staging` cannot tell them apart from real hardware.

7.6.1 Creating the virtual keyboard

```

1 let device = uinput::default()?
2   .name("hugo-phone-virtual-keyboard")?
3   .event(uinput::event::Keyboard::All)?
4   .create()?;

```

The constant `Keyboard::All` registers every key the kernel knows about. This matters for hot-reload: if we only registered the keys present in the initial *keymap*, switching the mapping to a new key would require destroying and recreating the device, which interrupts the flow.

7.6.2 Injecting a press

```

1 pub async fn press_dtmf(&mut self, dtmf: &str) -> Result<()> {
2     let Some(key) = self.keymap.get(dtmf) else {
3         warn!("DTMF {:?} unmapped", dtmf);
4         return Ok(());
5     };
6     self.device.press(key)?;
7     self.device.synchronize()?;
8     sleep(self.press_duration).await;
9     self.device.release(key)?;
10    self.device.synchronize()?;
11    Ok(())
12 }

```

Each press follows the cycle:

1. Look up the symbolic key matching the received DTMF.
2. Send a `press` event.
3. Call `synchronize()` so the kernel emits a `SYN_REPORT` and propagates the event to consumers.
4. Wait `press_duration` (typically 80 ms).
5. Send a `release` event and sync again.

The pause between *press* and *release* is important: too short and games like Hugo don't detect the press; too long and the game's engine triggers the action in continuous repetition.

7.7 The `watcher.rs` module

The configuration *hot-reload* lets you tweak the *keymap* and the press duration without restarting the service. It is particularly useful during initial calibration.

```

1 let mut watcher = notify::recommended_watcher(move |res: notify::Result<Event>| {
2     match res {
3         Ok(evt) => { let _ = tx.send(evt); }
4         Err(e) => error!("Watcher error: {}", e),

```

```
5     }  
6 })?;  
7 watcher.watch(parent, RecursiveMode::NonRecursive)?;
```

Two important details:

- We watch the **parent directory**, not the file itself. Many editors save a file by writing to a temporary and renaming it on top of the original (*atomic save*), which doesn't fire a *modify* event on the original file but a *create* in the directory.
- A 500 ms *debounce* is applied to avoid multiple reloads if the editor writes in several steps.

7.8 Practical verification

Test the bridge manually:

```
1 cd /opt/hugo-phone/bridge  
2 sudo ./hugo-bridge --config config.toml
```

Expected output:

```
INFO Config loaded from "/opt/hugo-phone/bridge/config.toml"  
INFO Virtual keyboard created  
INFO Watching ".../config.toml" for hot-reload  
INFO Connecting to AMI on 127.0.0.1:5038 ...  
INFO AMI banner: Asterisk Call Manager/9.0.0  
INFO Authenticated to AMI as hugo-bridge
```

Check that the virtual keyboard exists:

```
1 sudo apt install -y evtest  
2 sudo evtest
```

`hugo-phone-virtual-keyboard` must appear in the list. Select its number, then press DTMF keys in Linphone: `evtest` must show the corresponding press events.

7.9 The complete flow with the softphone

With the bridge running, DOSBox-Staging showing Hugo on the TV and Linphone registered:

1. Dial 9000 from Linphone.
2. Hear the beep in the earpiece.
3. Press 2. The character should jump/move up.
4. Try 4, 6, 8, 5.

In the bridge logs:

```
INFO DTMF 2 from 102  
INFO DTMF 4 from 102
```

7.10 Calibration

The most sensitive parameter is `press_duration_ms` in `config.toml`:

```
1 [bridge]
2 press_duration_ms = 80
```

Symptoms and adjustments:

Symptom	Adjustment
One press = several actions	Lower to 40–50 ms
The press doesn't register	Raise to 120–150 ms
Movement is choppy	Raise to 100 ms

Any change is applied on save (hot-reload), without restarting the service.

7.11 Enable as a service

Once validated manually:

```
1 sudo systemctl start hugo-bridge
2 sudo systemctl status hugo-bridge
3 sudo systemctl enable hugo-bridge
```

Live logs:

```
1 sudo journalctl -u hugo-bridge -f
```

7.12 Troubleshooting

7.12.1 Permission denied on `/dev/uinput`

- The service runs as `root`, this shouldn't happen.
- If you're running it manually, use `sudo`.
- Check the udev rule at `/etc/udev/rules.d/99-hugo-uinput.rules`.

7.12.2 The bridge connects but doesn't receive events

- Confirm that Linphone is calling 9000 and not some other extension.
- On the Asterisk console, does `UserEvent: HugoKey` appear when you press DTMF? If not, the problem is in the dialplan (Phase 2).

7.12.3 DTMFs reach the bridge but Hugo doesn't respond

- DOSBox-Staging requires window focus. If it was launched over SSH with X-forwarding, it loses focus.

- Test the injection against another application: with the bridge running, open `nano` on the DOSBox TTY and verify that the cursor moves when you press DTMF.

7.12.4 Cargo can't find dependencies during compilation

- Check Internet connectivity: `ping crates.io`.
- If DNS is restricted, configure:

```
1 echo "nameserver 1.1.1.1" | sudo tee /etc/resolv.conf
```

7.13 Phase wrap-up

With this phase, the whole software flow is validated. All that remains is replacing the Linphone softphone with a real analog telephone connected to the HT801.

Phase 5: Connecting the real telephone

8.1 Goal

Replace the softphone with a physical analog DTMF telephone, connected to the PBX through the Grandstream HT801 adapter. By the end of this phase, picking up the handset should take you directly into the game, without dialing any number.

8.2 Background concepts

8.2.1 What an FXS port is

An *FXS* (*Foreign eXchange Subscriber*) port is the kind of port the phone company provides to a residential user: it supplies about 48 V DC to power the phone, around 90 V AC to make it ring, and through *loop current* detects when the handset is picked up or hung up.

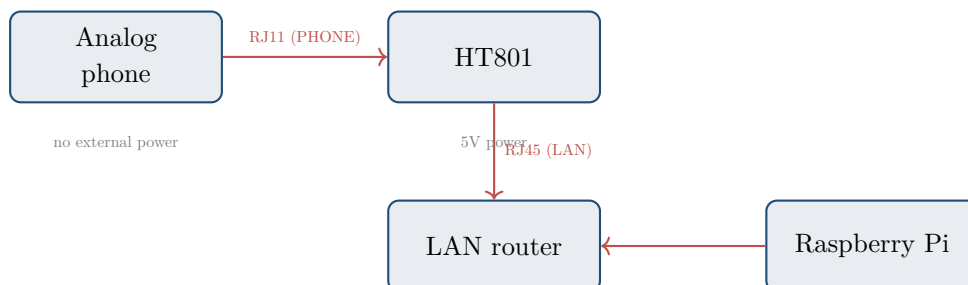
An *FXO* (*Foreign eXchange Office*) port is the opposite: it connects toward a telephone exchange and behaves like a phone.

The HT801 exposes an FXS port on its RJ11 connector, which lets you plug in any analog telephone as if the PBX were the phone company.

8.2.2 Auto-dial when going off-hook

The HT801's *Off-hook Auto-Dial* feature lets you set a number that will be dialed automatically when the handset is picked up. This is the key to replicating the Hugo experience: the user lifts the handset and is immediately playing, without dialing anything.

8.3 Physical connections



- The phone's RJ11 cable plugs into the port labeled **PHONE** (or **FXS**) on the HT801. *Not* the LAN port.
- The HT801 goes over Ethernet to the same switch/router as the Pi.

- The HT801's 5 V power supply plugs into the corresponding port.
- The analog phone needs no external power: the line provides voltage.

8.4 Step 1: Find the HT801's IP

The HT801 automatically obtains an IP via DHCP. There are two ways to learn it:

8.4.1 Method A: HT801's own IVR

1. Pick up the phone connected to the HT801.
2. Dial *** (three asterisks).
3. An English voice offers a diagnostic menu.
4. Dial 02 to hear the device's IP.

8.4.2 Method B: router panel

Enter the router's admin panel and look in the DHCP client list for a new device with a name similar to GS-HT801 or HT801-xxxx.

Once the IP is identified (let's say 192.168.1.11), **reserve** that IP for the HT801's MAC in the router, just like for the Pi.

8.5 Step 2: Access the HT801 web panel

From the Mac, open in a browser:

```
http://192.168.1.11
```

Credentials:

- **User:** admin
- **Password:** admin (older models) or the password printed on the device's bottom label (recent models with a unique per-unit password).

Warning

Change the default password before doing anything else: **Advanced Settings** → **Admin Password**.

8.6 Step 3: Configure the SIP account

Navigate to the FXS Port tab (or Profile 1 on some firmware versions).

In the codecs section:

- Choice 1: PCMU (G.711 μ -law)

Field	Value
Account Active	Yes
Primary SIP Server	192.168.1.10 (the Pi's IP)
SIP User ID	101
Authenticate ID	101
Authenticate Password	the secret for 101 in <code>sip.conf</code>
Name	Hugo phone 1
DTMF Payload Type	101
Preferred DTMF method	RFC2833 (priority 1)
SIP Registration	Yes
Outgoing call without registration	No
NAT Traversal	No

Table 8.1: HT801 SIP configuration.

- Choice 2: PCMA (G.711 A-law)
- The rest: None

8.7 Step 4: Configure Auto-Dial

On the same tab, find the `Offhook Auto-Dial` field and set it to:

```
9000
```

This causes the HT801 to automatically dial 9000 when the phone is picked up, which is the extension the dialplan ties to the game.

Click `Update` and then `Apply`. The HT801 restarts its SIP module and is registered within about 10 seconds.

8.8 Step 5: Verify the registration

On the Pi:

```
1 sudo asterisk -rvvv
```

```
hugo-pbx*CLI> sip show peers
Name/username      Host                Status
101/101            192.168.1.11       OK (5 ms)
102/102            (Unspecified)      UNKNOWN
```

Extension 101 must appear with the HT801's IP and status OK. If it says UNREACHABLE or the Host field is empty, check:

- Password match between the HT801 and `sip.conf`.
- That *Primary SIP Server* actually points at the Pi.
- Detailed logs:

```
1 hugo-pbx*CLI> sip set debug on
```

8.9 Step 6: The final test

All the pieces are wired together. The sequence:

1. Confirm DOSBox-Staging is showing Hugo on the TV.
2. Confirm the bridge service is active: `systemctl status hugo-bridge`.
3. **Pick up the phone.**
4. In the earpiece, you should hear the dialplan beep (after a brief moment while the HT801 dials 9000 internally).
5. Press **2**. Hugo jumps.
6. Press **4, 6, 8, 5** and play normally.
7. When you hang up, the call ends cleanly.

If everything works, the system is complete in its minimum configuration. The Hugo Phone project is alive.

8.10 Troubleshooting

8.10.1 The phone gives no dial tone when picked up

The HT801 isn't supplying line. Possible causes:

- FXS port LED in red: hardware or configuration error. Do a *factory reset* (physical Reset button pressed for 7 seconds).
- Bad or swapped RJ11 cable (try another one).
- Damaged telephone (try another phone).

8.10.2 There's dial tone but going off-hook doesn't dial 9000

Offhook Auto-Dial is misconfigured. Verify it is exactly 9000 with no spaces, asterisks or pound signs.

8.10.3 It connects but the beep is not heard

Codec problem:

- Make sure the HT801 prioritizes PCMU and PCMA.
- In the HT801, section *Audio Settings*, raise the *Tx* and *Rx* gains if volume is too low.

8.10.4 DTMFs don't produce action in Hugo

- Check the DTMF method in the HT801: it must be RFC2833.
- In `sudo asterisk -rvvv`, `UserEvent: HugoKey` must appear when you press digits.
- Some older HT801s have problems with RFC 2833 and require switching to SIP INFO. Try that method if the previous one doesn't work.

8.10.5 The call drops on its own after a few minutes

The dialplan has a 3600-second (1 hour) timeout in `Read()`. If the call drops earlier, a *session timer* is probably active. Check in `sip.conf` the line `session-timers=refuse`.

Phase 6: Multi-player mode (future)

9.1 Vision

The configuration described up to the previous chapter supports a single phone. The original television experience, however, contemplated multiple players calling at the same time, listening to the game's audio while waiting, and receiving rotating 30-to-60-second turns.

This phase describes the design needed to scale the system to that full experience. **It is not documented as a step-by-step executable guide** (that is work for a 2.0 version of the project), but as an architectural roadmap.

9.2 Hardware changes

- Replace the **HT801** (1 FXS port) with:
 - **HT802**: 2 FXS ports. Approx. cost \$55,000 CLP.
 - **HT814**: 4 FXS ports. Approx. cost \$110,000 CLP.
 - **Cisco SPA8000** (used): 8 FXS ports. Approx. cost \$80,000–120,000 CLP.
- Acquire as many additional analog phones as FXS ports you'll use.
- Optionally, a USB HDMI capture card to stream or record matches.

9.3 Asterisk changes

9.3.1 sip.conf: new extensions

Add extensions 102, 103, 104 according to the number of FXS ports, following the same pattern as the already documented 101.

9.3.2 confbridge.conf: conference rooms

Create the file `/etc/asterisk/confbridge.conf` with profiles for two kinds of participants:

```
1 [default_bridge]
2 type=bridge
3 max_members=10
4 mixing_interval=20
5
6 [player_active]
7 type=user
8 admin=no
9 marked=yes
```

```

10 quiet=yes
11
12 [player_waiting]
13 type=user
14 admin=no
15 marked=no
16 quiet=yes
17 music_on_hold_when_empty=no

```

9.3.3 extensions.conf: queue logic

Modify extension 9000 so that the first participant enters as *player_active* and subsequent ones as *player_waiting*:

```

1 [hugo-game]
2 exten => 9000,1,Answer()
3 same => n,Set(POSITION=${CONFBRIDGE_INFO(parties,hugo-room)})
4 same => n,GotoIf($[${POSITION} = 0]?active:waiting)
5 same => n(active),ConfBridge(hugo-room,default_bridge,player_active)
6 same => n(waiting),Playback(esperando-turno)
7 same => n,ConfBridge(hugo-room,default_bridge,player_waiting)

```

9.4 Turn logic

An additional component is needed (it could be another Rust process or a module inside the existing bridge) that every 60 seconds:

1. Identifies the current *player_active* via the `ConfbridgeList` event or `ConfbridgeList` queries over AMI.
2. Demotes them to *waiting* via `ConfbridgeMute` and a profile reconfiguration.
3. Promotes the next player in queue (FIFO) to *active*.
4. Announces the turn change by playing a pre-recorded message to everyone.

Only the active player should be able to control the character. This is achieved by having the bridge filter `UserEvent` messages by the `Channel` field and only accept those from the active channel.

9.5 Game audio in the conference

For waiting players to hear Hugo, the audio from DOSBox-Staging must be routed into the conference room.

9.5.1 Path A: ALSA loopback

```

1 sudo modprobe snd-aloop

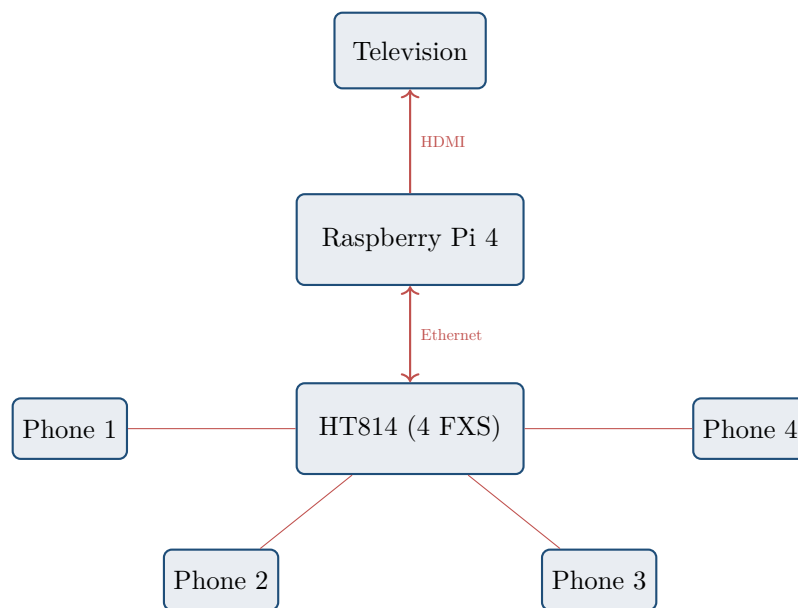
```

This creates a virtual device `hw:Loopback,0`. DOSBox-Staging is configured to output audio to that device. Asterisk opens the other end of the loopback as an input channel and *bridges* it into the room.

9.5.2 Path B: PipeWire

More flexible and modern but with higher resource usage. On the Pi 4 (2 GB) it could be a bottleneck; on Pi 5 it works without trouble.

9.6 Physical topology



9.7 Optional features

9.7.1 Player identification

At the start of every call, before entering the room, the player can be asked to dial a digit to identify themselves:

```
"Welcome to Hugo. Press 1 if you are new, or your player number  
if you already have one."
```

This makes it possible to keep a persistent ranking across sessions.

9.7.2 Scoreboard / ranking

A local SQLite database on the Pi can store scores per player. Each time Hugo registers a life lost or a level completed (information that could be captured via OCR on the HDMI output or by reading the DOSBox process memory), the scoreboard is updated.

9.7.3 On-screen scoreboard display

An overlay can be drawn on top of the HDMI output via:

- OBS Studio in composition mode.
- A second process drawing with SDL2 on a transparent layer.
- A small secondary LCD screen dedicated to the scoreboard.

9.7.4 Streaming to Twitch or YouTube

With a USB HDMI capture card plugged into the Mac (not the Pi, to avoid overloading it), matches can be broadcast. This lets other friends watch the game without being physically present.

9.8 Effort estimate

Component	Approx. effort
ConfBridge and new SIP extensions	4 hours
Turn logic (Rust)	16 hours
ALSA audio loopback	4 hours
Player identification	4 hours
SQLite persistence	4 hours
Scoreboard overlay	12 hours
Tests and calibration	8 hours
Estimated total	~52 hours

Table 9.1: Time estimate for implementing multi-player mode.

9.9 Wrap-up

The base system designed in chapters 4–8 is intended to grow into this multi-player version without rewriting components. When the time comes to implement it, a new detailed design phase should be opened taking these foundations as a reference.

Chapter 10

Technical reference

10.1 DTMF → keyboard mapping

DTMF	Frequencies (Hz)	Key	evdev code	Action in Hugo
1	697 + 1209	Q	KEY_Q	free / action 1
2	697 + 1336	↑	KEY_UP	jump or up
3	697 + 1477	E	KEY_E	free / action 2
4	770 + 1209	←	KEY_LEFT	left
5	770 + 1336	Space	KEY_SPACE	main action
6	770 + 1477	→	KEY_RIGHT	right
7	852 + 1209	Z	KEY_Z	free / action 3
8	852 + 1336	↓	KEY_DOWN	down / crouch
9	852 + 1477	X	KEY_X	free / action 4
*	941 + 1209	Left Shift	KEY_LEFTSHIFT	modifier
0	941 + 1336	Esc	KEY_ESC	pause / menu
#	941 + 1477	Enter	KEY_ENTER	confirm

Table 10.1: Default DTMF → keyboard mapping.

10.2 Key system files

File	Purpose
/etc/asterisk/sip.conf	Defines SIP extensions 101 and 102
/etc/asterisk/extensions.conf	Dialplan: logic for 9000
/etc/asterisk/manager.conf	Enables AMI with the hugo-bridge user
/etc/systemd/system/hugo-bridge.service	Bridge service unit
/etc/udev/rules.d/99-hugo-uinput.rules	Permissions on /dev/uinput
/opt/hugo-phone/bridge/hugo-bridge	Compiled bridge binary
/opt/hugo-phone/bridge/config.toml	Bridge configuration (hot-reload)
/opt/hugo-phone/games/hugo/	Game files
~/.config/dosbox/dosbox-staging.conf	Emulator configuration

10.3 Useful services and commands

10.3.1 Managing Asterisk

```
1 sudo systemctl status asterisk
2 sudo systemctl restart asterisk
3 sudo asterisk -rvvv           # interactive console
4 sudo asterisk -rx "reload"    # reload configs without restart
```

Commands inside the Asterisk CLI:

```
sip show peers           ; extension states
sip show registry       ; registered clients
sip set debug on/off    ; detailed traces
dialplan show hugo-game  ; show loaded dialplan
manager show settings   ; AMI state
manager show users       ; AMI users
manager show connected   ; active AMI sessions
core show channels       ; active calls
```

10.3.2 Managing the bridge

```
1 sudo systemctl status hugo-bridge
2 sudo systemctl restart hugo-bridge
3 sudo journalctl -u hugo-bridge -f      # live logs
4 sudo journalctl -u hugo-bridge -n 100  # last 100 lines
```

10.3.3 Rebuilding the bridge

```
1 cd /opt/hugo-phone/bridge
2 sudo -u beto ~/.cargo/bin/cargo build --release
3 sudo cp target/release/hugo-bridge ./hugo-bridge
4 sudo systemctl restart hugo-bridge
```

10.3.4 Inspecting the virtual keyboard

```
1 ls /dev/input/by-id/          # look for hugo-phone-...
2 sudo evtest                   # select the device
3                               # watch events when pressing DTMF
```

10.3.5 HT801 health check

```
1 ping 192.168.1.11             # IP reachability
2 nmap -p 5060 192.168.1.11     # SIP port open
```

10.4 Structure of the config.toml file

```
1 [ami]
2 host = "127.0.0.1"
3 port = 5038
4 username = "hugo-bridge"
5 secret = "REAL-PASSWORD"
6
7 [bridge]
8 press_duration_ms = 80
9 event_name = "HugoKey"
10 key_field = "Key"
11
12 [keymap]
```

```
13 "2" = "Up"
14 "4" = "Left"
15 "6" = "Right"
16 "8" = "Down"
17 "5" = "Space"
18 "0" = "Esc"
19 "1" = "Q"
20 "3" = "E"
21 "7" = "Z"
22 "9" = "X"
23 "*" = "LeftShift"
24 "#" = "Enter"
```

10.5 Supported key names

The keyboard.rs parser recognizes:

- **Directional:** Up, Down, Left, Right
- **Special:** Space, Enter, Esc (or Escape), Tab, Backspace
- **Modifiers:** LeftShift, RightShift, LeftCtrl, RightCtrl, LeftAlt, RightAlt
- **Letters:** A through Z
- **Function keys:** F1 through F12

If an unknown name is assigned, the bridge logs a warning but keeps running.

10.6 AMI protocol anatomy

10.6.1 Initial banner

```
Asterisk Call Manager/9.0.0
```

10.6.2 Login

```
Action: Login
Username: hugo-bridge
Secret: REAL-PASSWORD
Events: user,call
(blank line)
```

10.6.3 Successful login response

```
Response: Success
Message: Authentication accepted
(blank line)
```

10.6.4 UserEvent produced by the dialplan

```
Event: UserEvent
Privilege: user,all
UserEvent: HugoKey
Channel: PJSIP/101-00000003
Caller: 101
Key: 2
Uniqueid: 1715641234.3
(blank line)
```

10.7 Glossary

AMI *Asterisk Manager Interface*. Plain-text TCP protocol for controlling Asterisk from external clients.

ATA *Analog Telephone Adapter*. Device that converts an analog line to SIP.

DOSBox Emulator of the MS-DOS operating system and the IBM PC compatible hardware. DOSBox-Staging is a modernized fork.

DTMF *Dual-Tone Multi-Frequency*. Signaling system using pairs of tones, used by telephone keypads.

evdev Linux kernel subsystem that exposes input devices to *user space*.

FXO *Foreign eXchange Office*. Port that connects to a switchboard as if it were a telephone.

FXS *Foreign eXchange Subscriber*. Port that supplies line as if it were the phone company.

KMSDRM *Kernel Mode Setting / Direct Rendering Manager*. Linux kernel layer that allows drawing directly to the screen without an X server.

PBX *Private Branch Exchange*. Private telephone switchboard.

RFC 2833 Standard for carrying DTMF tones inside RTP packets, out of band relative to the main audio.

RTP *Real-time Transport Protocol*. Protocol for transmitting audio and video in real time, over UDP.

SIP *Session Initiation Protocol*. Signaling protocol for initiating, maintaining and terminating voice/video sessions.

uinput Linux kernel subsystem that lets *user space* create virtual input devices.

10.8 External references

- Official Asterisk documentation: wiki.asterisk.org
- DOSBox-Staging: dosbox-staging.github.io
- Rust uinput crate: docs.rs/uinput

- Tokio: tokio.rs
- Grandstream HT801 manual: grandstream.com/products
- Raspberry Pi documentation: raspberrypi.com/documentation
- Hugo on MyAbandonware: myabandonware.com/game/hugo-eya

10.9 Credits

Hugo, the character and the franchise, are the property of **ITE Media ApS** (originally *Interactive Television Entertainment*), Denmark. This project is a non-profit technical tribute with no official affiliation with the company.

The complete system, its code and its documentation were developed as a personal project by **Beto** in collaboration with Claude (Anthropic) during May 2026.

“Hugo, watch out for that rock!”
— *Generations of Chilean children in the nineties*